CS 499 Milestone Two: Enhancement One – Software Design and Engineering Narrative

Jared Borck

CS 499 Computer Science Capstone

September 20, 2026

Artifact Description

The artifact I selected for my Software Engineering and Design enhancement is **Travlr Getaways**, a full-stack web application that I originally created in CS 465: Full Stack Development during the summer of 2026. The application was built using Angular for the frontend, Node.js and Express for the backend and REST API, and MongoDB for data storage. In the original version, users could view available trips, while authenticated users could access the administrative functionality used to add, edit, and delete trip information.

I selected Travlr Getaways because it gave me a complete application that I could improve instead of starting a new project from the beginning. It already had several connected parts, including frontend components, services, API routes, authentication, backend controllers, and MongoDB models. This made it a good project for demonstrating that I can understand how an existing system works and then expand it without breaking the functionality that is already there.

For this enhancement, I added a customer rating and review system. Users can now view reviews for individual trips and submit ratings from one to five in five categories: value, destination, overall trip quality, accommodations, and activities. They can also leave an optional written review. A user must be logged in before submitting a review, and each authenticated user can submit only one review for a particular trip. Users can edit or delete their own reviews but cannot modify reviews that belong to someone else.

Justification for Inclusion

I chose to include Travlr Getaways in my ePortfolio because the enhanced version demonstrates more of my software development abilities than the original project did. The original application already showed that I could build a working full-stack application, but this enhancement required me to work with an

existing architecture and add new functionality across several parts of the system while keeping the original features working.

One of the biggest improvements involved authentication and authorization. Before this enhancement, the application mainly checked whether someone was logged in before giving them access to trip-management features. Once I added customer reviews, that was no longer enough because a regular customer should be able to submit a review without also gaining permission to modify trips. I expanded the user model to support customer and administrator roles and included the user's role in the JSON Web Token. New accounts are given the customer role by default, while trip-management actions are restricted to administrators. I also added an Angular route guard so that non-administrative users cannot access the Add Trip or Edit Trip pages through the interface.

Another important part of the enhancement was making sure security did not depend only on hiding buttons in the frontend. The backend now checks that a user has the administrator role before allowing them to create, update, or delete trip records. For reviews, the application gets the user's ID from the authenticated JWT instead of trusting a user ID sent from the browser. Before a review can be edited or deleted, the backend checks whether the current user actually owns that review. This means that even if someone tries to call the API directly, they still cannot modify another user's review.

The review system also required several new backend and frontend pieces. I created a review model that stores the trip code, user information, five rating categories, optional written review, and timestamps. I added controller logic and REST API routes for retrieving, creating, updating, and deleting reviews. The backend validates that every rating is a whole number from one through five, and it also prevents the same user from submitting more than one review for the same trip.

On the Angular side, I created a review model, expanded the existing data service with methods for the review API, and created a new trip review component. Anyone can view the reviews for a trip, while

logged-in users can submit their own ratings. The page also checks which review belongs to the current user so that the Edit Review and Delete Review buttons only appear on that person's review.

I also improved how the application handles changes in login state. During testing, I noticed that logging in changed the navigation bar correctly, but logging out could leave the Add, Edit, and Delete buttons visible until the page was refreshed. The buttons would no longer actually work because the user was logged out, but the interface was still showing them. I changed the authentication service to publish authentication and administrator state so the other components could respond immediately. After that change, the controls appeared and disappeared correctly without requiring a refresh.

These changes demonstrate my ability to work across a full-stack application using Angular, Node.js, Express, MongoDB, REST APIs, and JWT-based authentication and authorization. More importantly, they show that I can take an existing application, find areas that need improvement, and make connected changes across the frontend and backend instead of treating each part of the program separately.

Reflection on the Enhancement Process

This enhancement helped me better understand the difference between adding a feature that appears to work and designing a feature that works correctly under different conditions. Adding a basic review form would not have been very difficult by itself. The harder part was making sure it worked correctly with authentication, different user roles, ownership rules, validation, and the trip functionality that was already in the application.

One of the main challenges was separating customer permissions from administrator permissions. The original application did not have much reason to distinguish between different types of authenticated users because logging in was mainly connected to the administrative side of the application. Once customers needed accounts for reviews, simply checking whether someone was logged in was not enough anymore. A customer needed permission to create and manage their own reviews without also being

allowed to add, edit, or delete trips. Adding customer and administrator roles solved that problem and made the permissions much clearer.

Another challenge was making sure review ownership was actually enforced instead of just making the interface look secure. I tested the API directly through PowerShell using different JWTs. I tried having a normal customer edit another user's review and received a 403 response, which confirmed that the backend ownership check was working. I also tested whether a customer could perform an administrator-only trip operation and received another 403 response. Along with those security tests, I tested creating, retrieving, updating, and deleting reviews and tried submitting a duplicate review to make sure the application rejected it.

Testing the finished interface also caused me to change part of my original design. At first, I made the written review required. After using the feature, I decided that someone should be able to rate a trip without being forced to write a comment. I changed the Angular form, backend validation, and MongoDB model so the written section became optional while the five numerical ratings remained required. That change showed me why testing a program from the user's point of view is important. A feature can work exactly as it was originally designed and still be improved after actually using it.

There were also a few smaller problems I had to work through. One was the login-state issue where some administrator controls stayed visible after logout until the page refreshed. I also had to make sure the new review API worked correctly before connecting it to the Angular interface. Testing the API separately first made debugging easier because I could tell whether a problem was coming from MongoDB, Express, authentication, or the frontend instead of trying to troubleshoot everything at once.

This enhancement most directly demonstrates **Course Outcomes 4 and 5**. I believe this enhancement fully demonstrates Outcome 4 because I used full-stack development tools and established software engineering techniques to add meaningful functionality to an existing application. The enhancement

required changes to the frontend, REST API, backend logic, authentication system, authorization rules, and database model. It was not just an isolated change to one part of the project.

I also believe this enhancement fully demonstrates Outcome 5 because security was considered throughout the design instead of being added only after the feature was finished. Role-based authorization protects administrative trip operations, review ownership is checked on the backend, authenticated JWT information is used to identify users, rating values are validated, and duplicate reviews are prevented. These controls help prevent users from accessing functionality or changing data that they should not have permission to modify.

This enhancement gives less direct evidence for Course Outcomes 1, 2, and 3 on its own. The communication and collaboration outcomes are demonstrated more directly through my code review, written narratives, and final ePortfolio. The algorithmic portion of Outcome 3 will be addressed more directly in my next enhancement, where I plan to use the rating information created during this milestone to calculate scores and rank trips. Keeping that work separate allowed this enhancement to stay focused on software engineering, application design, and security.

Overall, the enhanced Travlr Getaways application is much more complete than the original version. Instead of only displaying trips and allowing authenticated users to manage trip information, the application now supports customer interaction through a secure rating and review system. Working through this enhancement gave me more experience maintaining and expanding an existing full-stack application and helped reinforce the importance of authentication, authorization, validation, testing, and designing software around different types of users.